Continual Learning: Why Models Forget, and What We Can Do About It

Sagi Shaier

About Me

Sagi Shaier

Currently a visiting professor at Johannes Gutenberg University Mainz.

Background

PhD (2025): Factual Knowledge-Enhanced Question Answering in Dynamic Environments.

Current focus

Sparsity (Mixture-of-Experts), continual learning, Biomedical AI, drawing inspiration from biological systems, such as the human brain.

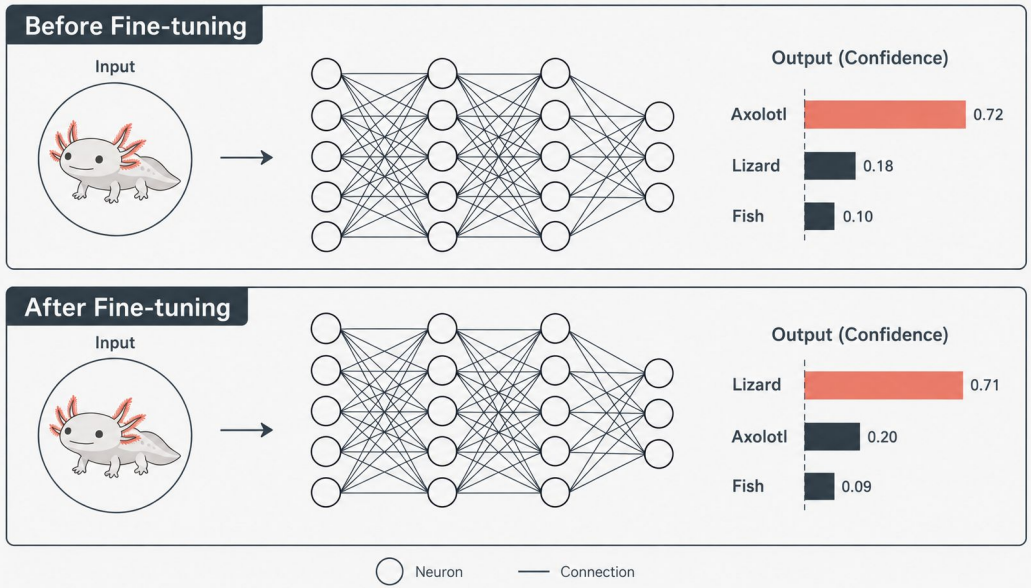


Forgetting in AI

How Do We Define It?

How Do We Define It?

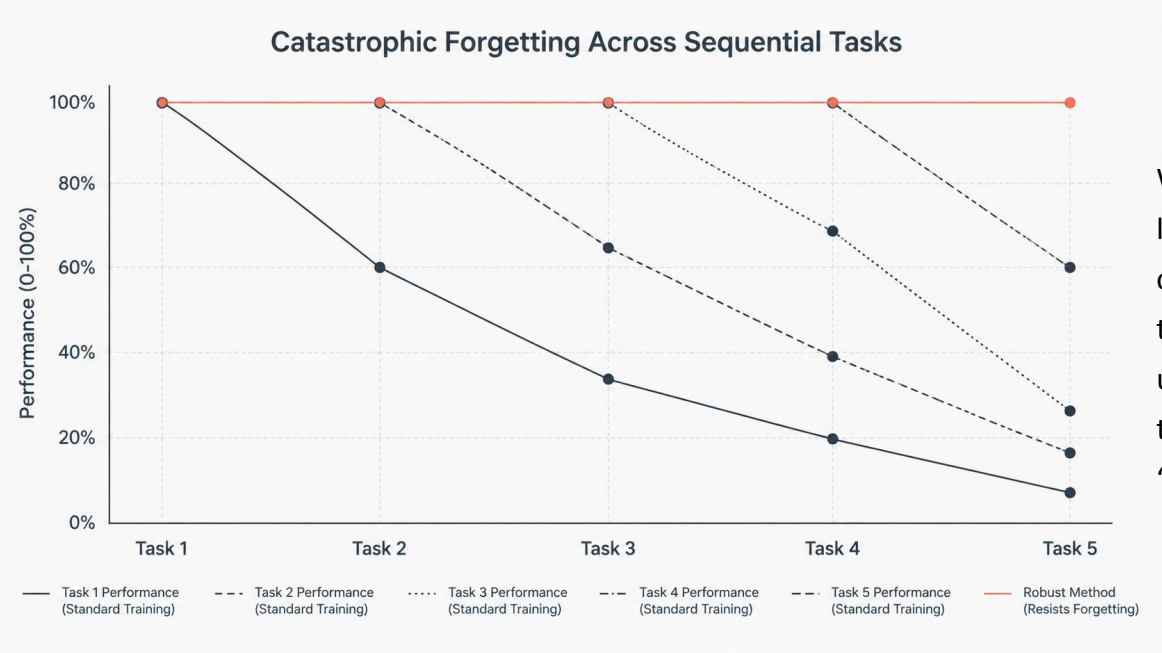
A model losing previously learned information, typically because new training overwrites or interferes with the parameters that encoded that earlier knowledge.



Fine-tuning, continued pretraining, instruction tuning, and many other training procedures are all ways of changing parameters - and each one carries this risk.

How Do We Evaluate It?

We measure a model's performance on a set of benchmarks before training on a new task, then compare it to its performance on the same benchmarks after that training. A drop in performance is evidence of forgetting.



When the drop is large, the literature calls it catastrophic forgetting - though there's no universally agreed threshold for how large “catastrophic” has to be.

An Open, Unsolved Problem

Despite how central this problem is to ML, it's not fully understood and far from being solved.

An Open, Unsolved Problem

Anatomy of Catastrophic Forgetting: Hidden Representations and Task Semantics

Does it depend on specific layers in the network?

An Open, Unsolved Problem

Anatomy of Catastrophic Forgetting: Hidden Representations and Task Semantics

What Happens To BERT Embeddings During Fine-tuning?

Does it depend on the specific training paradigm, such as fine-tuning?

An Open, Unsolved Problem

Anatomy of Catastrophic Forgetting: Hidden Representations and Task Semantics

What Happens To BERT Embeddings During Fine-tuning?

Wide Neural Networks Forget Less Catastrophically

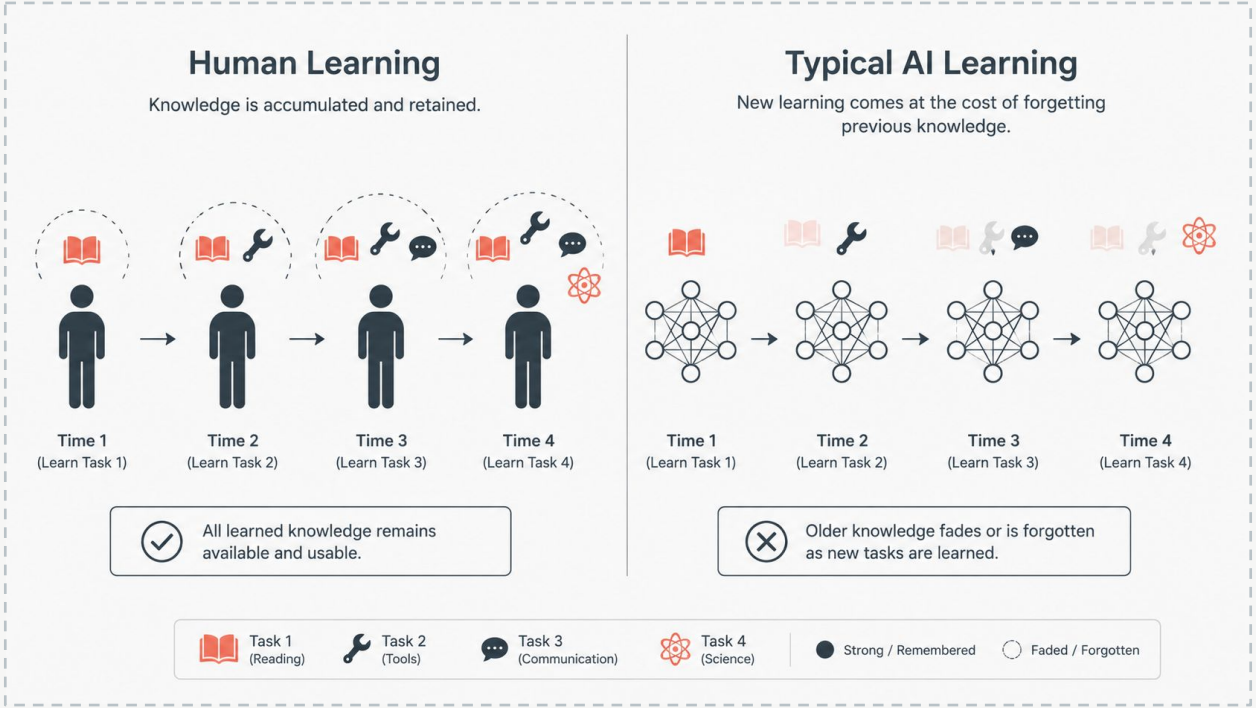
Does it depend on the specific architecture features, such as network width?

The Field of Continual Learning

Continual learning focuses on developing models that can keep learning over time - without forgetting what they already know.

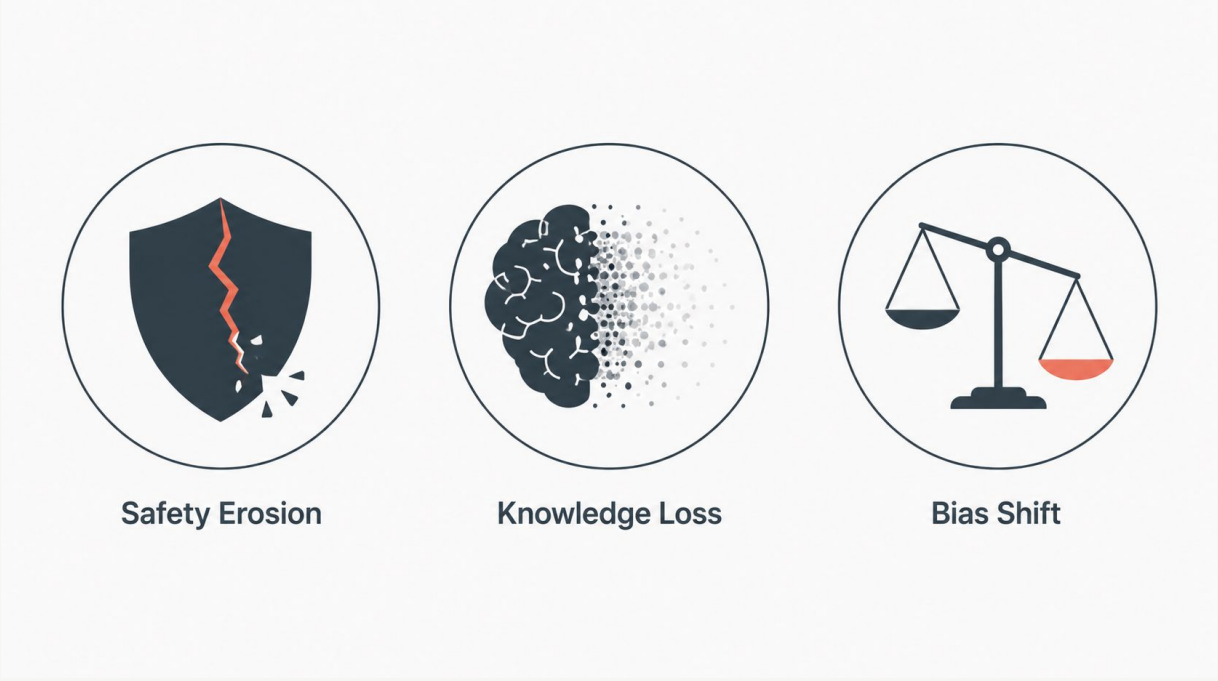
The Field of Continual Learning

Why does this matter?



Failure Cases: Why Continual Training Is Risky

Before looking at solutions, it's worth seeing what goes wrong in practice when we keep updating a model's weights.



FAILURE MODE 1

Safety Alignment Eroded by Fine-Tuning

FAILURE MODE 1

Safety Alignment Eroded by Fine-Tuning

Fine-tuning a model can make it less safe.

FAILURE MODE 1

Safety Alignment Eroded by Fine-Tuning

Removing RLHF Protections in GPT-4 via Fine-Tuning

Shows that fine-tuning lets attackers remove GPT-4's RLHF safety protections using as few as 340 adversarial examples.

FAILURE MODE 1

Safety Alignment Eroded by Fine-Tuning

Removing RLHF Protections in GPT-4 via Fine-Tuning

Shadow Alignment: The Ease of Subverting Safely-Aligned Language Models

Fine-tuning on just 100 malicious examples is enough to make a safely aligned LLMs generate harmful content.

FAILURE MODE 1

Safety Alignment Eroded by Fine-Tuning

Removing RLHF Protections in GPT-4 via Fine-Tuning

Shadow Alignment: The Ease of Subverting Safely-Aligned Language Models

Fine-Tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend It!

Shows that the safety alignment of LLMs can be compromised by fine-tuning with only a few adversarially examples.

FAILURE MODE 1

Safety Alignment Eroded by Fine-Tuning

Removing RLHF Protections in GPT-4 via Fine-Tuning

Shadow Alignment: The Ease of Subverting Safely-Aligned Language Models

Fine-Tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend It!

Attack via Overfitting: 10-Shot Benign Fine-Tuning to Jailbreak LLMs

Even 10 non-adversarial examples can break models' refusal behavior

FAILURE MODE 1

One Pattern

LLMs safety alignment can be broken down with very little data.

The takeaway: safety alignment is not a separate, protected layer of the model. It is encoded in the same weights as everything else, which means ordinary fine-tuning can overwrite it.

FAILURE MODE 2

Loss of Factual and General Knowledge

FAILURE MODE 2

Loss of Factual and General Knowledge

Fine-tuning doesn't just risk safety - it can erode a model's factual and general knowledge: the kind of “common knowledge” we would assume is stable.

FAILURE MODE 2

Loss of Factual and General Knowledge

An Empirical Study of Catastrophic Forgetting in LLMs During Continual Fine-Tuning

Shows that continual instruction tuning can cause catastrophic forgetting of domain knowledge, reasoning, and reading comprehension.

FAILURE MODE 2

Loss of Factual and General Knowledge

An Empirical Study of Catastrophic Forgetting in LLMs During Continual Fine-Tuning

LoRAMoE: Alleviate World Knowledge Forgetting in LLMs

Finds that instruction tuning can damage the LLMs' world knowledge.

FAILURE MODE 2

The Pattern Repeats

Similarly to what we saw with safety alignment: knowledge that feels permanent and stable is stored in the same weights which can be overwritten unexpectedly.

There is nothing structurally protecting “common knowledge” from being pushed aside by whatever the model is trained on next.

FAILURE MODE 2 → 3

Knowledge Isn't the Only Thing That Shifts

If factual knowledge can be overwritten, what about something less concrete - a model's apparent values and stances?

FAILURE MODE 3

Political / Ideological Bias Shift

FAILURE MODE 3

Political / Ideological Bias Shift

PoliTune: Analyzing the Impact of Data Selection and Fine-Tuning on Economic and Political Biases in LLMs

Shows that parameter-efficient fine-tuning (PEFT) on ideologically filtered data can steer a model's political views left or right.

FAILURE MODE 3

Political / Ideological Bias Shift

PoliTune: Analyzing the Impact of Data Selection and Fine-Tuning on Economic and Political Biases in LLMs

On the Relationship between Truth and Political Bias in Language Models

Finds that optimizing reward models for truthfulness can result in left-leaning political biases

FAILURE MODE 3

Same Mechanism, Different Symptom

Whether it's a safety refusal, a fact, or a political stance, the underlying story is the same: it's a pattern encoded in the weights, and training can move those weights without anyone necessarily intending it to.

QUICK RECAP

What's Actually Happening

The model's weights change - through fine-tuning, continued pretraining, or instruction tuning.

Old knowledge gets overwritten inside the parameters themselves.

QUICK RECAP

What's Actually Happening

The model's weights change - through fine-tuning, continued pretraining, or instruction tuning.

Old knowledge gets overwritten inside the parameters themselves.

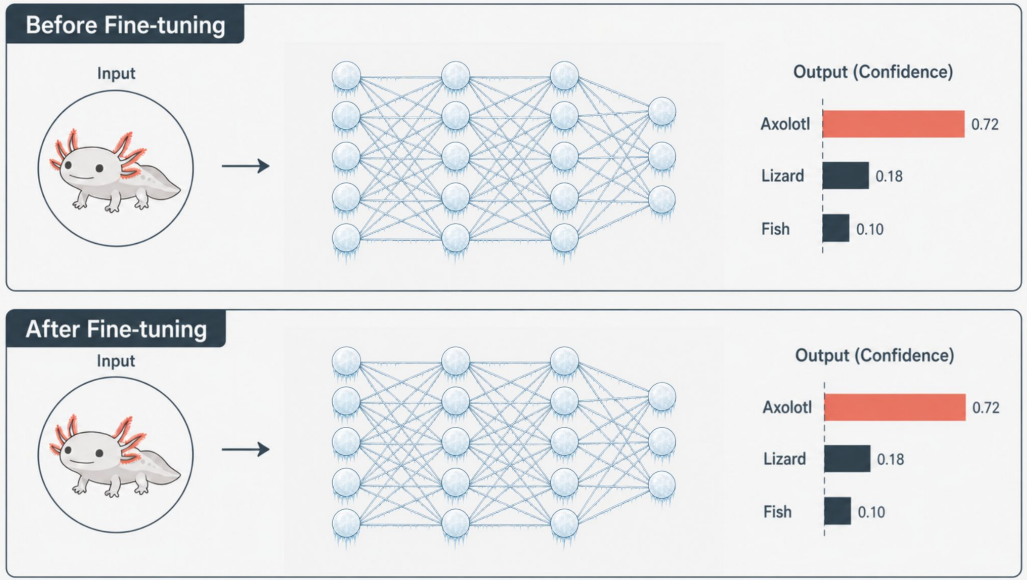
Relevant to anyone who fine-tunes or continue-trains a model themselves.

PART 2 - SOLUTIONS

The Obvious “Fix”

Just... Don't Update the Weights?

If updating the weights is what causes forgetting, why not just... not update them?



THE OBVIOUS “FIX”

Just... Don't Update the Weights?

Problem:

A frozen model can never learn anything new - no new domains, no facts past the training cutoff, no adapting to a specific task or dataset.



Just... Don't Update the Weights?

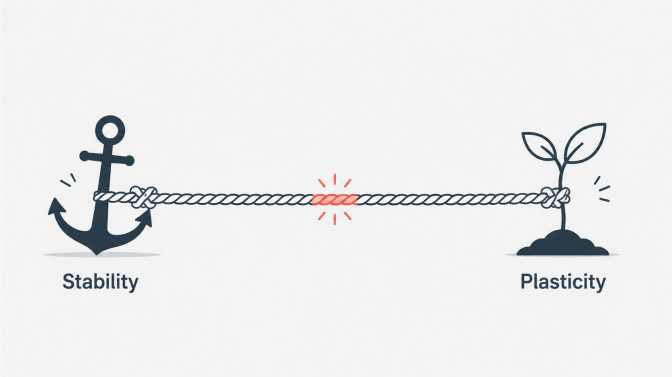
You may already be thinking: “but can't the model adapt without updating weights, just through the prompt?”

Yes - that's in-context learning, and it's exactly the second half of this talk. Hold on to that thought.

So We're Stuck

The model needs to change in order to learn, but changing is what causes it to forget.

This tension is the stability-plasticity dilemma.



CORE CONCEPT

Stability-Plasticity Dilemma

Stability

The ability to retain (i.e., not forget) previously learned knowledge.

Plasticity

The ability to acquire new knowledge.

A Balancing Act

A system that's too stable can't learn anything new; a system that's too plastic overwrites what it already knew.

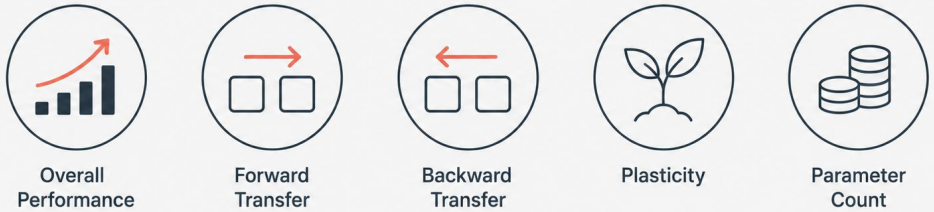
A Balancing Act

A system that's too stable can't learn anything new; a system that's too plastic overwrites what it already knew.

Every continual learning method is, at its core, a different way of trading off between these two.

Beyond Just Measuring Forgetting

There are several other metrics we care about when studying continual learning:



MECHANISM

Mechanistically: Why Weight Updates Cause Forgetting

MECHANISM

Catastrophic Interference

MECHANISM

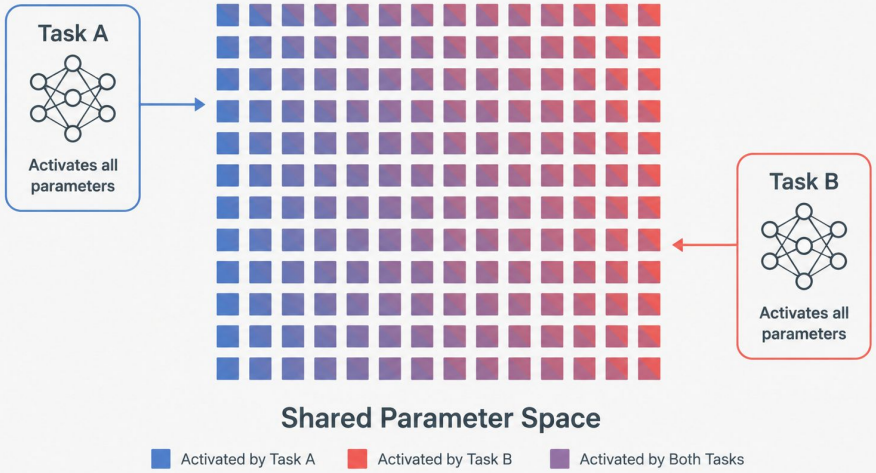
Catastrophic Interference

Updating weights for a new task can destroy weights the old task depended on.

MECHANISM

Shared Weights

A single network encodes everything in the same parameters, so there's no dedicated, protected space for old knowledge. Because the old and new tasks share those parameters, updating weights for the new task pulls them away from the values the old task needed, causing interference.

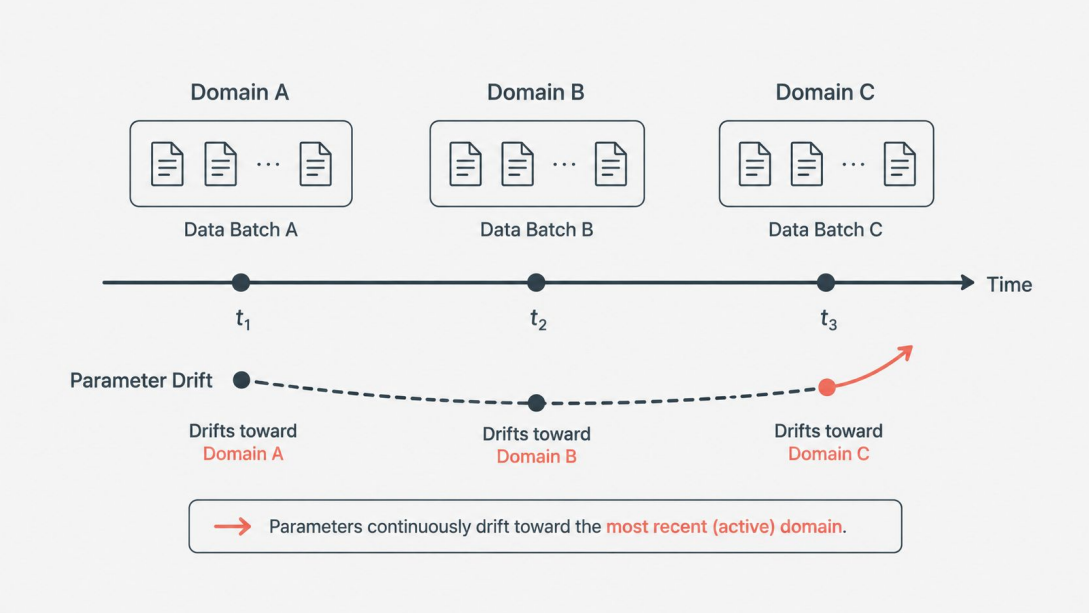


Different Distributions Updates

MECHANISM

Different Distributions Updates

Training on sequential data - this domain, then that domain - rather than a uniform mix of everything at once causes parameters to drift toward whatever the model is currently seeing.



RELEVANCE CHECK

Why This Isn't Just Old ML Theory

RELEVANCE CHECK

Why This Isn't Just Old ML Theory

Modern LLM workflows hit the same problem at scale.

RELEVANCE CHECK

Why This Isn't Just Old ML Theory

→ **Continued pretraining on a new domain corpus.**

RELEVANCE CHECK

Why This Isn't Just Old ML Theory

→ **Continued pretraining on a new domain corpus.**

→ **Standard fine-tuning on a downstream task.**

RELEVANCE CHECK

Why This Isn't Just Old ML Theory

→ **Continued pretraining on a new domain corpus.**

→ **Standard fine-tuning on a downstream task.**

→ **Instruction tuning itself, the step that turns a base model into a helpful assistant.**

RELEVANCE CHECK

Why This Isn't Just Old ML Theory

→ **Continued pretraining on a new domain corpus.**

→ **Standard fine-tuning on a downstream task.**

→ **Instruction tuning itself, the step that turns a base model into a helpful assistant.**

→ **Parameter-efficient methods like LoRA still exhibit catastrophic forgetting, despite touching far fewer parameters.**

PART 3 - MITIGATIONS

What Can We Do About It?

Three Families of Methods

MITIGATIONS

Three Families of Methods

Replay

Mix old data back in while training on new data.

Regularization

Add a constraint to the objective function to limit how much a model can change.

Architectural

Give new knowledge its own dedicated space in the network.

Three Families of Methods

Replay · Regularization · Architectural

Practical note: these methods generally require access to the model and training loop, which is not something usually achievable through a hosted API.

FAMILY 1

Replay

What If We Just Retrained on Everything?

If models forget when we train them on new data, what if we just… retrained on all the data, old and new, every time?

Why Full Retraining Doesn't Work

Slow

Billions to trillions of tokens.

Expensive

Lots of GPUs, much money.

Costly to the planet

Significant water and energy use.



A Cheaper Idea: Mix in *Some* Old Data

While training on new data (fine-tuning, etc.), keep mixing in some samples of the old data, so the model doesn't drift entirely away from what it knew before. The idea is to approximate and recover the old data distributions.



A Cheaper Idea: Mix in *Some* Old Data

Keep mixing in some samples of the old data while training on new data.

This can work quite well!

A Cheaper Idea: Mix in *Some* Old Data

Keep mixing in some samples of the old data while training on new data.

This can work quite well!

But… requires access to examples of the original training distribution.

This Raises Important Questions

This Raises Important Questions

How much data?

How many examples per task or distribution? We want enough that the model doesn't forget, but not so many that it becomes prohibitive.

This Raises Important Questions

How much data?

How many examples per task or distribution? We want enough that the model doesn't forget, but not so many that it becomes prohibitive.

Which data?

Do we select random examples? Or something else?

A Practical Example

iCaRL: Incremental Classifier and Representation Learning

A Practical Example

iCaRL: Incremental Classifier and Representation Learning

How much data: fixed budget K. each task gets K/t examples. t is the number of tasks observed so far.
So the total memory stays fixed, and the per-task allocation shrinks automatically as more tasks arrive, rather than a fixed number of examples being set per task in advance.

A Practical Example

iCaRL: Incremental Classifier and Representation Learning

How much data: fixed budget K. each task gets K/t examples. t is the number of tasks observed so far.
So the total memory stays fixed, and the per-task allocation shrinks automatically as more tasks arrive, rather than a fixed number of examples being set per task in advance.

Which data: pick examples closest to the feature mean of the class/task.

A Variation: Generate Instead of Store

One cool variation: maybe you don't need to store previous examples at all

A Practical Example

Continual Learning with Deep Generative Replay

Instead of storing real examples from old tasks, they train a generative model to produce synthetic examples that approximate the old task's data distribution.

Why This Helps

This sidesteps the need to store a growing number of previous tasks data, which matters when storage or privacy limits make keeping the original examples impractical.

The Tradeoff

The generator itself has to be trustworthy: if it drifts or degrades, the replayed examples stop representing what the model originally knew, and the previous data, and the replay signal becomes noisy.

FAMILY 2

Regularization

Which Parameters Actually Matter?

We said models forget because parameters change and new information overwrites existing ones. So… what if we knew which parameters are the most important – the ones we definitely want to keep?

Which Parameters Actually Matter?

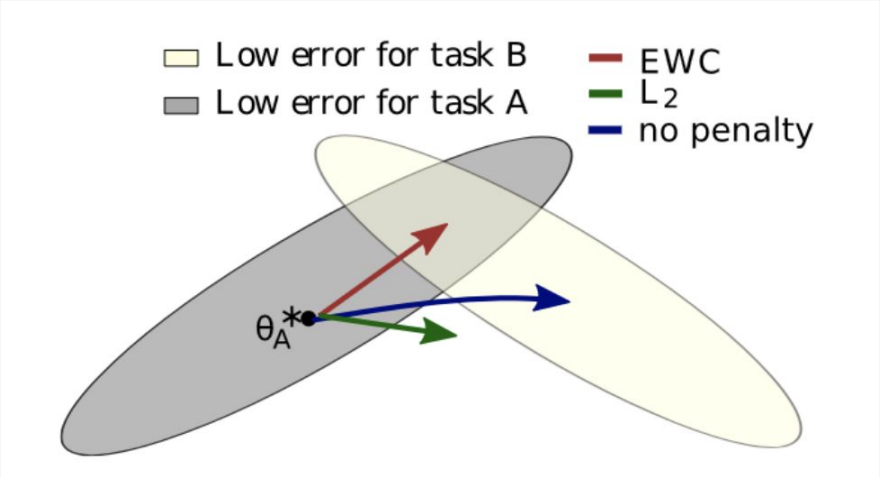
Idea:

Let the model update its weights, but penalize changes to the parameters that matter most for previously learned tasks by adding an additional term to the objective function.

Classic Examples

Overcoming catastrophic forgetting in neural networks

Elastic Weight Consolidation (EWC): identifies which weights were important for prior knowledge using the diagonal of the Fisher Information Matrix. Then adds a penalty term during training on a new task, discouraging large moves away from those important weights.



Classic Examples

Overcoming catastrophic forgetting in neural networks

Continual Learning Through Synaptic Intelligence

Unlike EWC, which estimates parameter importance from the Fisher information at the end of training, SI accumulates importance online throughout training based on each parameter's contribution to loss reduction.

FAMILY 2: REGULARIZATION

Classic Examples

Overcoming catastrophic forgetting in neural networks

Continual Learning Through Synaptic Intelligence

Memory Aware Synapses: Learning what (not) to forget

Unlike SI, which measures each parameter's contribution to reducing the loss, MAS measures how sensitive the network's output is to a small change in that parameter, so it needs no labels or loss (so unsupervised).

Function Regularization

Regularization strategies can also target the model's output or intermediate representations instead of its weights directly.

Function Regularization

Regularization strategies can also target the model's output or intermediate representations instead of its weights directly.

One example of this is knowledge distillation: the previously trained model acts as a teacher, where the currently trained model is the student. The loss penalizes the student's predictions for drifting away from the teacher's predictions on some data.

Function Regularization

Regularization strategies can also target the model's output or intermediate representations instead of its weights directly.

One example of this is knowledge distillation: the previously trained model acts as a teacher, where the currently trained model is the student. The loss penalizes the student's predictions for drifting away from the teacher's predictions on some data.

The general idea stays the same: add some term or constraint to an objective function that limits how much a model can change, in order to prevent unwanted drift.

FAMILY 3

Architectural

Beyond a Fixed Architecture

So far, both replay and regularization operate within a fixed network architecture: the number of parameters and their connectivity stay constant, only the parameter values change over training.

But what if it didn't have to?

Give New Knowledge Its Own Space

Idea:

Rather than controlling how existing parameters change (regularization) or rehearsing old data to protect them (replay), architectural methods change the model's structure itself: allocating new parameters for new tasks, or isolating and freezing parameters tied to old tasks so they cannot be overwritten.

FAMILY 3: ARCHITECTURAL

Architectural Sub-Strategies

Parameter allocation

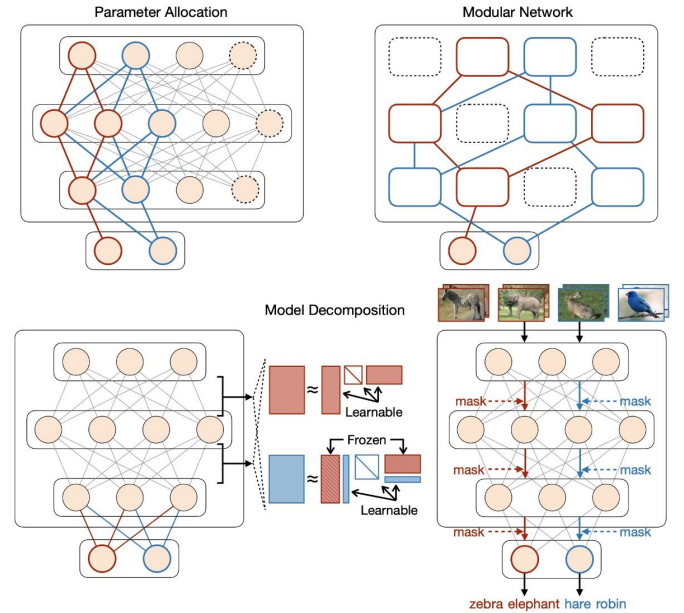
A dedicated portion of the network's parameters set aside for each individual task. Parameters can be reused by later tasks, but they are frozen.

Model decomposition

Splits the architecture into a task-sharing component that continues to be updated across all tasks, and separate task-specific components.

Modular network

Uses separate sub-networks to learn each new task, without pre-defining task-sharing or task-specific components.



A Comprehensive Survey of Continual Learning: Theory, Method and Application. Liyuan Wang, Xingxing Zhang, Hang Su, Jun Zhu, Fellow, IEEE

PEFT as Partial Isolation

LoRA and other parameter-efficient fine-tuning (PEFT) methods are a partial version of the isolation principle: the base model's weights are frozen, and only a small set of newly introduced parameters are trained.

PEFT as Partial Isolation

LoRA and other parameter-efficient fine-tuning (PEFT) methods are a partial version of the isolation principle: the base model's weights are frozen, and only a small set of newly introduced parameters are trained.

In continual learning specifically, O-LoRA* extends this by giving each new task its own LoRA parameters.

The idea is to freeze the LoRA parameters from all previous tasks, and also to add an orthogonal constraint between the new parameters and the frozen ones, so each task's subspace doesn't interfere with previously learned ones.

* Orthogonal Subspace Learning for Language Model Continual Learning

MITIGATIONS

Three Families of Methods

Replay

Mix old data back in while training on new data.

Regularization

Add a constraint to the objective function to limit how much a model can change.

Architectural

Give new knowledge its own dedicated space in the network.

RECAP

Three Families of Methods

Replay · Regularization · Architectural

But, what's actually done in practice?

IN PRACTICE

Frontier Models Have Many Training Phases

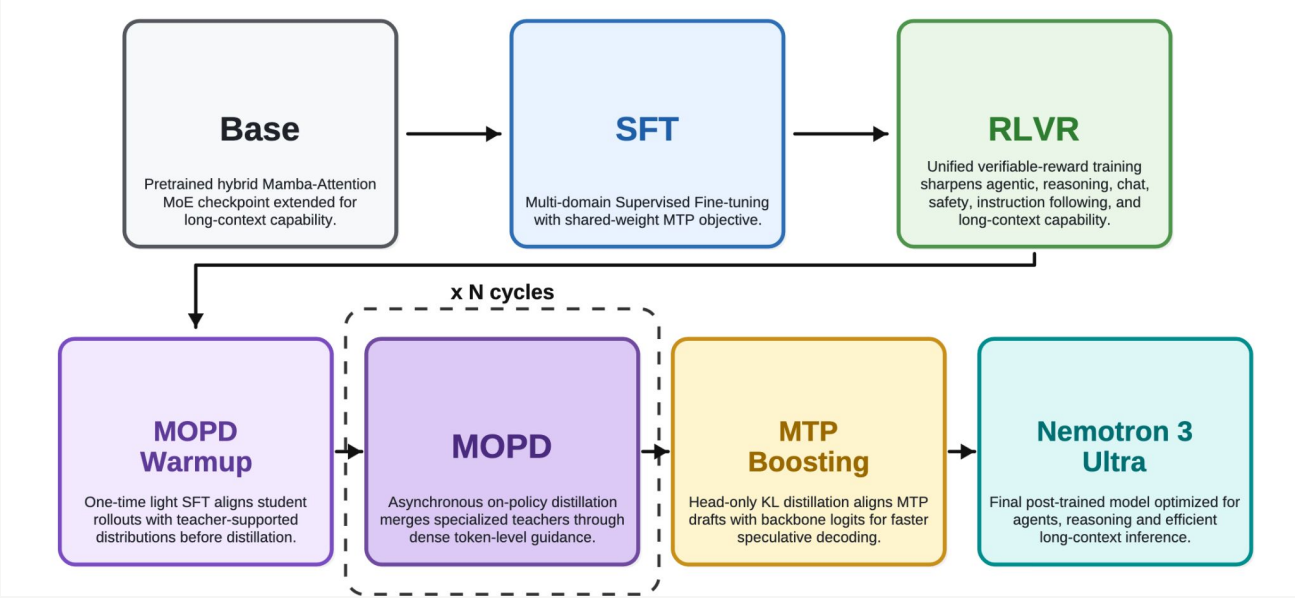
Frontier models (Claude, ChatGPT, Llama, Gemini, and others) go through many distinct training phases: pretraining, SFT, multiple stages of RL, etc.

IN PRACTICE

Frontier Models Have Many Training Phases

Nvidia

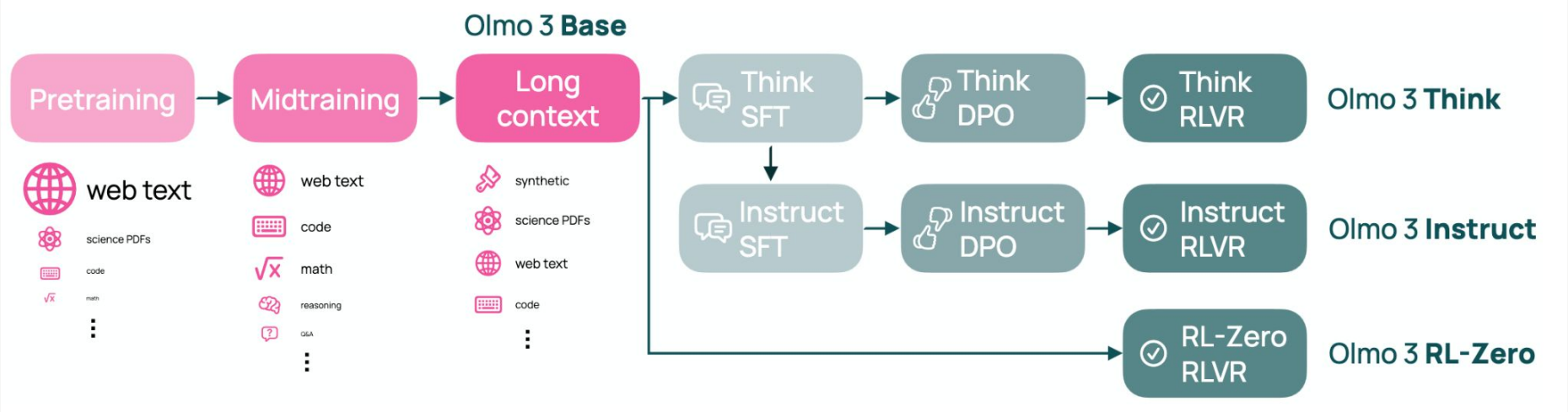
“Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning”



IN PRACTICE

Frontier Models Have Many Training Phases

Allen Institute for AI
"Olmo 3"

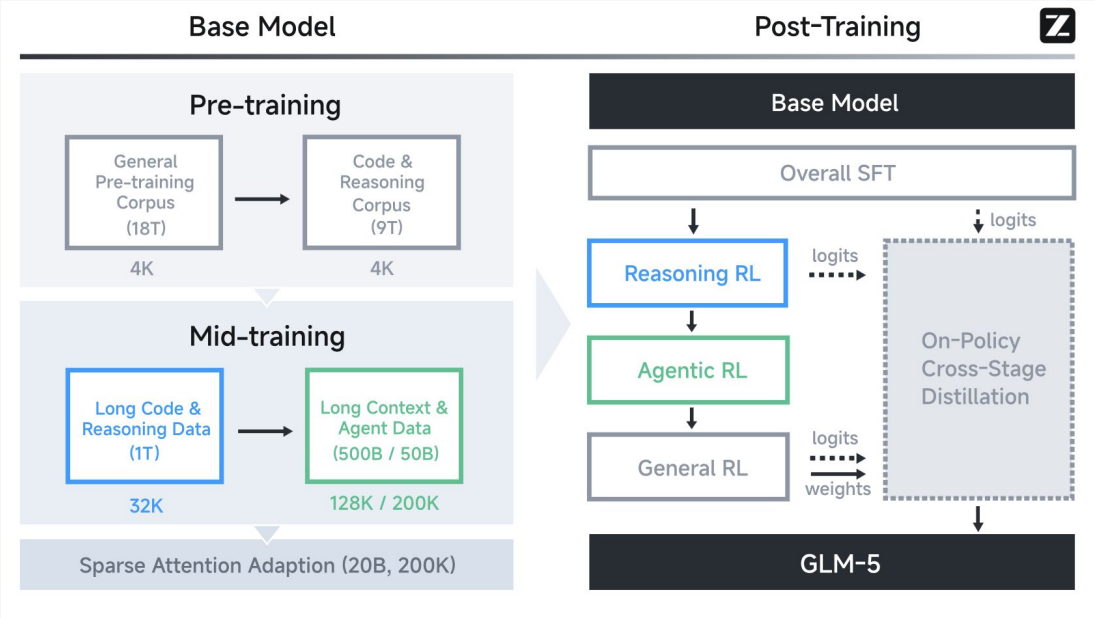


IN PRACTICE

Frontier Models Have Many Training Phases

GLM-5 Team

"GLM-5: from Vibe Coding to Agentic Engineering"



Forgetting Can Strike at Any Phase

Forgetting, or divergence, can happen in any of the phases!

How Do Different Labs Tackle This?

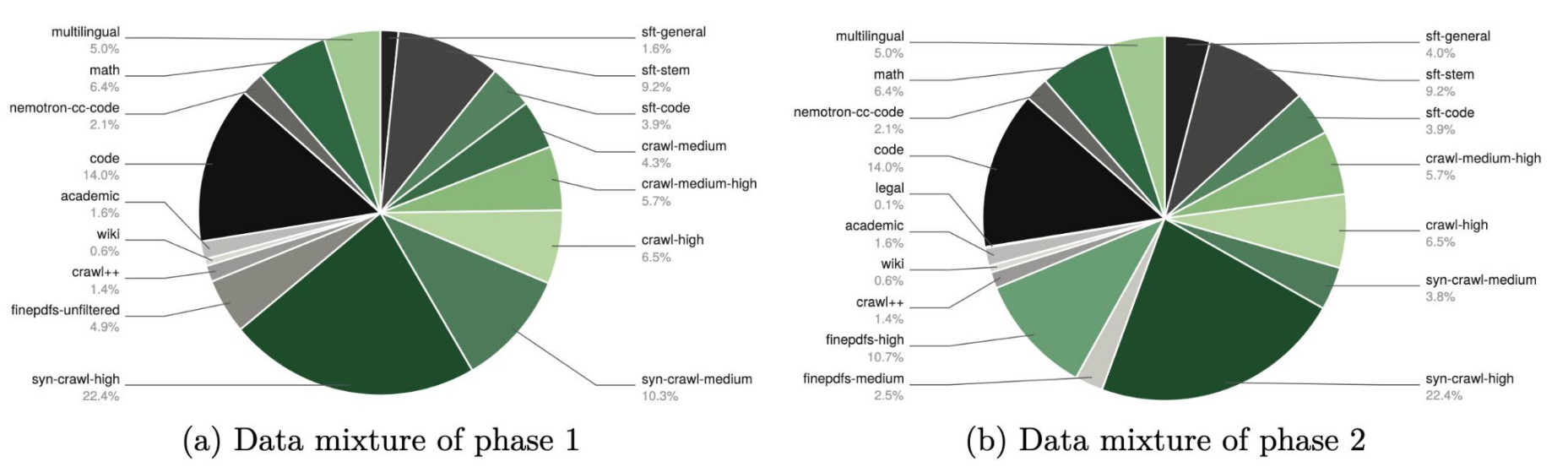
Let's look at a few examples.

HOW LABS TACKLE THIS

NVIDIA / Nemotron

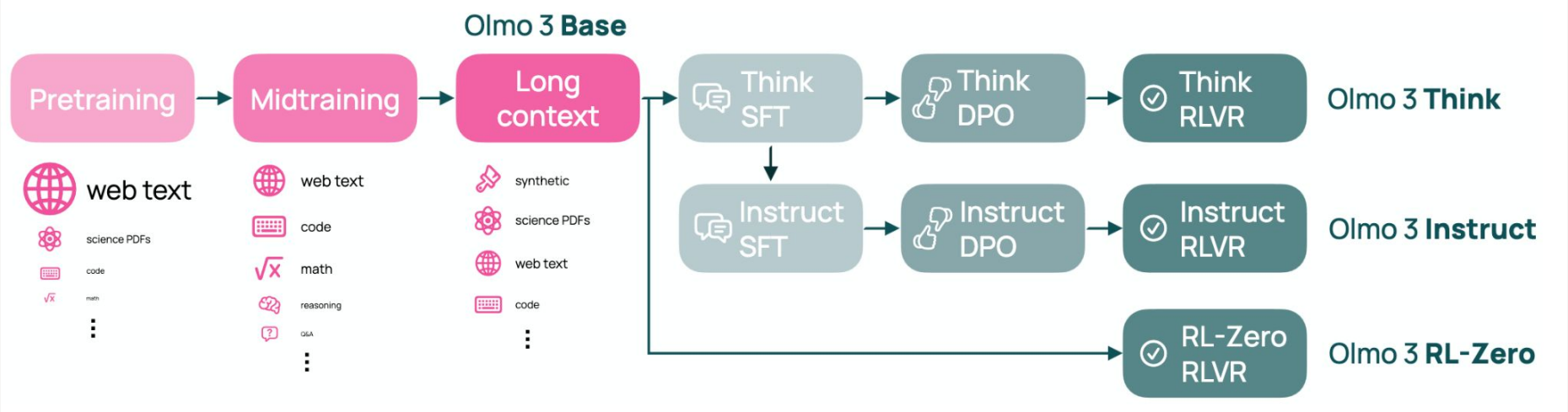
Architecture	MoE with 512 experts per layer, 22 activated.
Replay	Some similar training datasets between phases.
Regularization	On-policy distillation constrain how far the student can drift.

NVIDIA / Nemotron



Phase 1 is biased for diversity and phase 2 is biased for quality.

Allen Institute for AI / “Olmo 3”



GLM

Architecture

MoE (256 experts, 744B total / 40B active).

Replay/ Regularization

On-policy distillation with training prompts from previous phases.

The teacher rewards the student when it generates probable outputs and penalizes it when it doesn't.

One Taxonomy With Many Decisions

The replay, regularization, and architecture taxonomy isn't applied once.

Each new phase is an opportunity to overwrite what the model learned in the phase before it.

It's a set of decisions that practitioners make over and over again at nearly every phase.

PRACTICAL TAKEAWAYS

Putting This Into Practice

Putting This Into Practice

→ If you're just prompting a model via API or chat, without training anything, none of the mitigation methods (regularization, replay, architectural) apply to you.

This entire toolkit only becomes relevant once you're updating weights.

Putting This Into Practice

→ *If you're just prompting a model via API or chat, without training anything, none of the mitigation methods above (regularization, replay, architectural) apply to you. This entire toolkit only becomes relevant once you're updating weights.*

→ **But if you are updating the weights (pretraining, fine-tuning, etc.), you don't have to pick just one method.**

Putting This Into Practice

→ *If you're just prompting a model via API or chat, without training anything, none of the mitigation methods above (regularization, replay, architectural) apply to you. This entire toolkit only becomes relevant once you're updating weights.*

→ *But if you are updating the weights (pretraining, fine-tuning, etc.), you don't have to pick just one method.*

→ **Full regularization methods (EWC-style) generally assume you can compute per-parameter importance scores, which is probably out of reach for most users working with very large models.**

Putting This Into Practice

→ *If you're just prompting a model via API or chat, without training anything, none of the mitigation methods above (regularization, replay, architectural) apply to you. This entire toolkit only becomes relevant once you're updating weights.*

→ *But if you are updating the weights (pretraining, fine-tuning, etc.), you don't have to pick just one method.*

→ *Full regularization methods (EWC-style) generally assume you can compute per-parameter importance scores, which is probably out of reach for most researchers working with very large models.*

→ **Replay is realistic for almost anyone: if you're fine-tuning on your domain data, mix in general-purpose or instruction data alongside it.**

Low effort and can reduce the drift. The big labs do this. Try a few different data mixes and compare, rather than committing to a single fine-tuning run.

Putting This Into Practice

→ *If you're just prompting a model via API or chat, without training anything, none of the mitigation methods above (regularization, replay, architectural) apply to you. This entire toolkit only becomes relevant once you're updating weights.*

→ *But if you are updating the weights (pretraining, fine-tuning, etc.), you don't have to pick just one method.*

→ *Full regularization methods (EWC-style) generally assume you can compute per-parameter importance scores, which is probably out of reach for most researchers working with very large models.*

→ *Replay is realistic for almost anyone: if you're fine-tuning on your domain corpus, deliberately mix in some general-purpose or instruction data alongside it.*

Low effort, meaningfully reduces drift. The big labs do this. Try a few different data mixes and compare, rather than committing to a single fine-tuning run.

→ **LoRA and other PEFT methods are the practical default for most users fine-tuning open-source LLMs.**

Cheap, works on modest hardware, and gives partial protection against forgetting by design, since most of the base model's weights are never touched.

Two Habits Worth Keeping

1) **Keep checkpoints** along the way, not just of the final model, so you can roll back if you notice degradation.
2) **Evaluate continuously** during training, not just on your target task, but on a collection of general benchmarks too. This is a main method to see whether the model is diverging as training progresses.

Non-Parametric “Forgetting”

A Familiar Experience

Have you ever told an LLM something at the start of a conversation, “always respond concisely,” “always respond in bullet points,” and found it stopped following that instruction later on?

A Familiar Experience

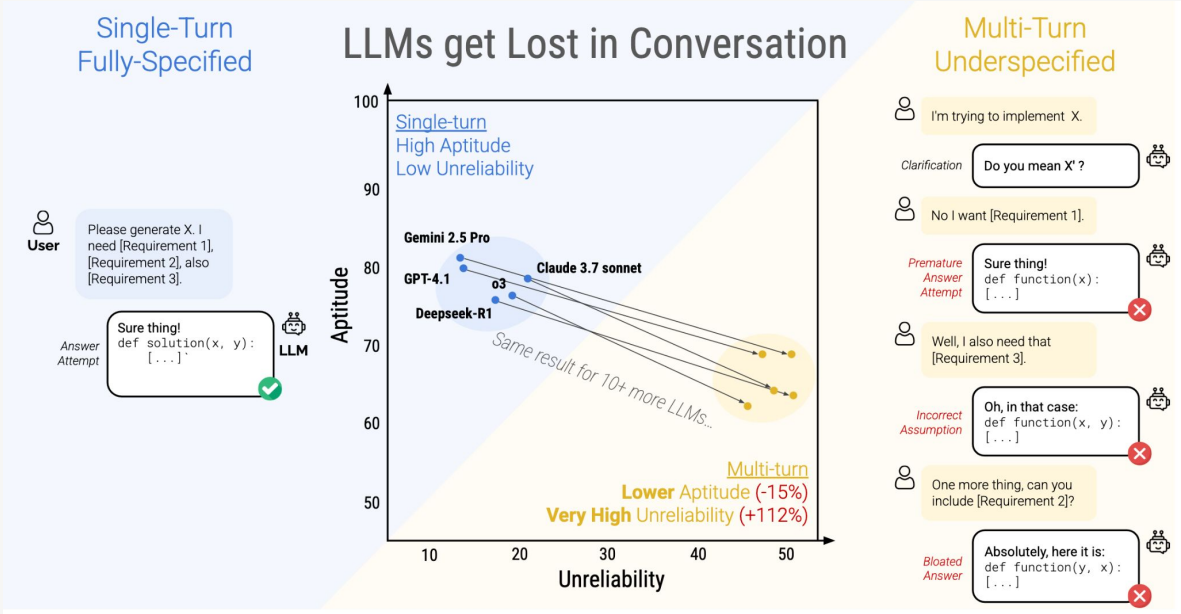
If you think about what we've covered so far, this is quite interesting: the weights are not actually changing.

This Is Real!

Lost in Multi-Turn Conversation

“LLMs Get Lost in Multi-Turn Conversation”

Finds that all of the top open and closed LLMs show lower performance in multi-turn conversations over single-turn.



Aptitude is defined as performance in the best-case conversation simulation.

Unreliability is defined as the gap between best- and worst-case performance.

How Is This Different?

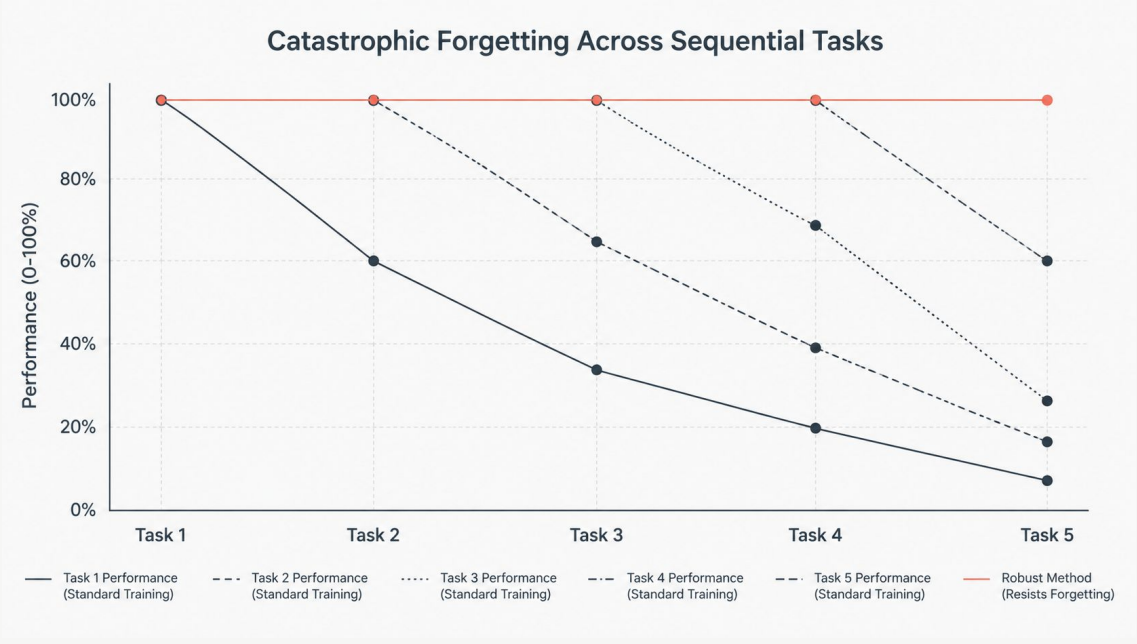
The weights stay completely frozen. Old information is still technically present, in a long conversation or document, and also in the model's weights, but the model stops using it effectively.

How Is This Different?

Relevant to anyone who just uses a model via API or chat, no training required to hit this.

Quick Recap

We defined forgetting in AI as a performance drop on tasks the model previously handled correctly.



The Puzzle

Now, if we have the same model, and the same information in the context, and we just add to it (assuming it stays within the model's context length), why would the model perform differently, appearing to “forget” information from the text?

Why “Forgetting” Can Appear Even When Weights Don't Change

Position Isn't Neutral: “Lost in the Middle”

Position Isn't Neutral: “Lost in the Middle”

“Lost in the Middle: How Language Models Use Long Contexts”

Performance degradation when the position of the relevant information changes.

Establishes that models don’t treat context positions symmetrically.



Order Matters

Order Matters: Same Facts, Different Order, Different Answer

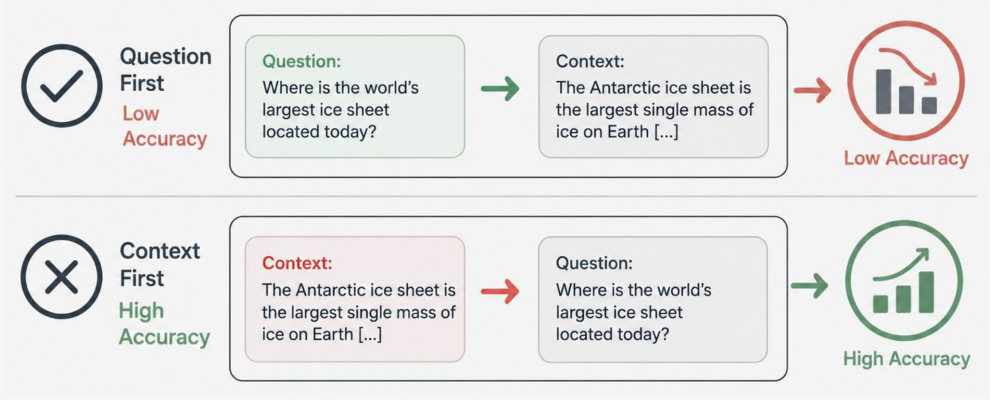
The same information, presented in a different order, can change the model's answer.

“Premise Order Matters in Reasoning with Large Language Models”

LLMs are surprisingly brittle to the ordering of the premises in reasoning tasks.

“It is not about what you say, it is about how you say it: A surprisingly simple approach for improving reading comprehension”

Question vs. context first change reading-comprehension performance.



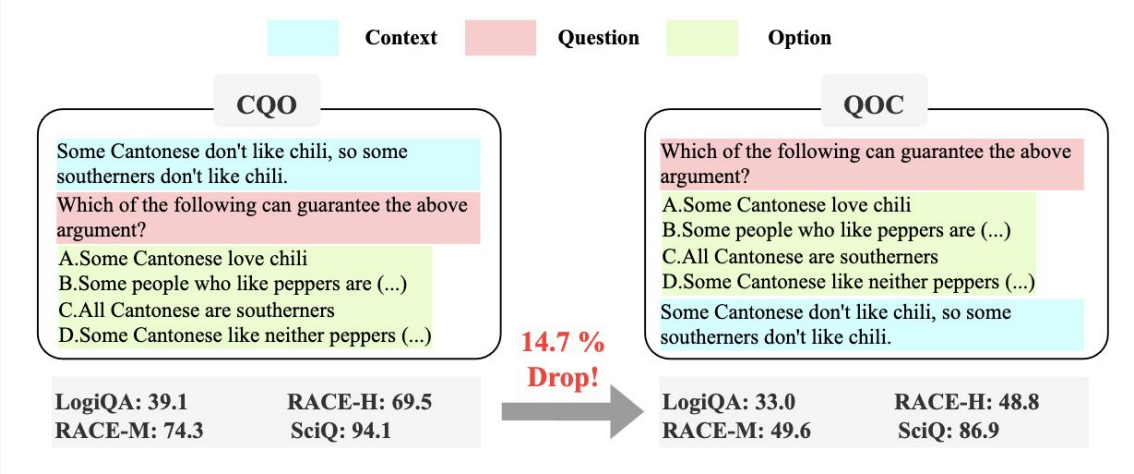
DEEPER MECHANISM

Mechanistically: Why “Forgetting” Can Appear Even When Weights Don't Change

“Lost in the Prompt Order: Revealing the Limitations of Causal Attention in Language Models”

An architectural cause:

In decoder-only (causal) models, if the question or answer options come before the context, the causal mask creates a situation where these tokens cannot attend forward to the context.



DEEPER MECHANISM

Forgetting in In-Context Continual Learning

“Understanding Generalization and Forgetting in In-Context Continual Learning”

Shows that the attention mechanism inherently causes interference, which shows up as both order sensitivity and forgetting. Even without any parameter updates.

Forgetting in In-Context Continual Learning

“Understanding Generalization and Forgetting in In-Context Continual Learning”

Shows that the attention mechanism inherently causes interference, which shows up as both order sensitivity and forgetting. Even without any parameter updates.

1) Forgetting depends on task order.
2) Forgetting gets worse when the tasks are similar.

Geometric Explanation

“Measuring and Controlling Instruction (In)Stability in Language Model Dialogs”

Found that models show a significant drift away from an initial instruction within just 8 rounds of conversation!

Geometric Explanation

“Measuring and Controlling Instruction (In)Stability in Language Model Dialogs”

Geometric explanation: attention decay.

- System prompt's tokens always stay in a cone in representation space.
- The cone stays stable when the model generate its own text and there’s no user input.
- But, new user turns can widen the cone.

As it widens, the prompt's tokens make up a smaller percentage of the space.

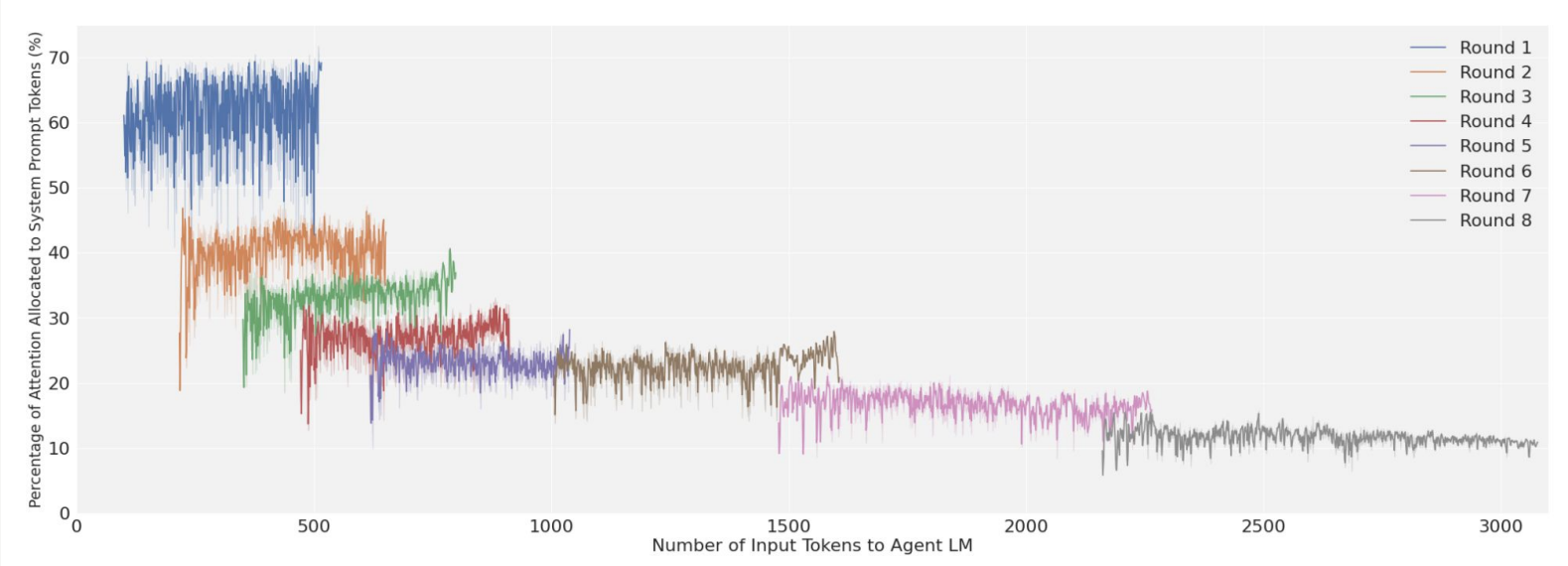
The tokens are still present, they just occupy smaller space of it.

Geometric Explanation

“Measuring and Controlling Instruction (In)Stability in Language Model Dialogs”

As it widens, the prompt's tokens make up a smaller percentage of the space.

The tokens are still present, they just occupy smaller space of it.



MITIGATIONS, CONTINUED

What To Do About It

What To Do About It

Fresh conversation

The most direct fix is simply not letting a conversation run long enough to drift. Starting a new conversation instead of continuing an old one can mitigate the cone-widening and lost-in-the-middle effect, at the cost of losing whatever context isn't explicitly re-supplied.

What To Do About It

Fresh conversation

The most direct fix is simply not letting a conversation run long enough to drift. Starting a new conversation instead of continuing an old one can mitigate the cone-widening and lost-in-the-middle effect, at the cost of losing whatever context isn't explicitly re-supplied.

Positional and structural fixes

Put critical information at the start or end of context, not buried mid-prompt.

Order context before the question when using causal/decoder models.

What To Do About It

Fresh conversation

The most direct fix is simply not letting a conversation run long enough to drift. Starting a new conversation instead of continuing an old one avoids the cone-widening effect, at the cost of losing whatever context isn't explicitly re-supplied.

Positional and structural fixes

Put critical information at the start or end of context, not buried mid-prompt.
Order context before the question when using causal/decoder models.

Recap & consolidation

Ask the model to summarize everything so far, then start a fresh conversation with that summary as a single-turn prompt. This is what tools like Claude's “/compact” are doing under the hood.

What To Do About It

Fresh conversation

The most direct fix is simply not letting a conversation run long enough to drift. Starting a new conversation instead of continuing an old one avoids the cone-widening effect, at the cost of losing whatever context isn't explicitly re-supplied.

Positional and structural fixes

Put critical information at the start or end of context, not buried mid-prompt.
Order context before the question when using causal/decoder models.

Recap & consolidation

Ask the model to summarize everything so far, then start a fresh conversation with that summary as a single-turn prompt. This is what tools like Claude's "/compact" are doing under the hood.

Task and topic separation

Since interference is worst between similar tasks, keeping distinct sub-tasks in separate conversations or contexts can mitigate the negative-transfer.

PRACTICAL TAKEAWAYS

What This Means for You

What This Means for You

Unlike parametric forgetting, none of this requires access to a training loop. It's entirely about how you structure prompts and manage conversation state.

That makes it actionable for anyone just calling an API, which is probably most users.



ONE MORE THING

Practical Hazard

ONE MORE THING

The Model Can Change Under You

When you don't have access to the actual model weights, certain unexpected things can happen that make it appear as if the model has suddenly gotten worse.

If you're running a long-term study using a hosted API rather than a local checkpoint, the model version can be updated by the provider without your knowledge, which can contaminate your results.

ONE MORE THING

Two Ways This Shows Up

→ **The model version changes.**

→ **Some providers route weaker or cheaper models to a subset of queries automatically.**

Adjacent, But Distinct From Forgetting

This isn't the model forgetting anything, it's the model itself changing under the hood.

ONE MORE THING

Practical Mitigation

Pin to a specific model version or snapshot when the provider offers one, and log the exact model identifier used for every run, not just “GPT-4” or “Claude,” but the specific dated version.

WRAPPING UP

Summary of Key Takeaways

Two Failure Modes With One Underlying Tension

Parametric forgetting

The parameters change. Old knowledge gets genuinely overwritten. The stability-plasticity dilemma.

Non-parametric forgetting

Weights are frozen. Old information is still present in both the model and the context. But the model just stops using it effectively. Attention-driven interference.

Same Symptom With Different Mechanism

Both can be seen as “forgetting” in the functional sense: performance on something the model previously handled correctly degrades.

But the mechanism, and therefore the fix, is completely different in each case.

Different Fixes for Different Mechanisms

Parametric fixes:

Replay

Mix old data back in while training on new data.

Regularization

Add a constraint to the objective function to limit how much a model can change.

Architectural

Give new knowledge its own dedicated space in the network.

Different Fixes for Different Mechanisms

Non-parametric fixes:

Short/fresh conversations, positional structuring, recap and consolidation, task separation.

DISCUSSION

Closing: Questions to Sit With

DISCUSSION

Question 1

What do you think forgetting looks like in the human brain?

- Is it that your brain literally no longer has that information (weights changed and overwritten)?
- Or is it that you simply can't retrieve it anymore, the tip-of-the-tongue feeling where you know you know it, but you can't “find it” (a retrieval problem)?

DISCUSSION

Question 2

If forgetting also happens in the human brain, maybe it’s actually beneficial?

Maybe the question is not so much “how to not forget”, but more along the lines of “what should we forget”.

Can you think about times/situations when forgetting is needed? Or when we want models to forget? Or what should they forget?

DISCUSSION

Question 3

Can you think about a way to develop different models or mechanisms that can better handle the stability-plasticity dilemma?

Thanks for Listening

Questions?